SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I _Thamaraiselvi S (192511042)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Thamaraiselvi S

Course Faculty

Annexure A

DESIGN TASK

Q1. Predictive Parser Design (Application Level)

Task:
Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.
.
Grammar:
E → T E'
E' → + T E' | ε
T → F T'
T' → * F T' | ε
F → (E) | id
Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

Test Case:
Input String:
id + id * id
Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted**

Q2. Shift-Reduce Parsing (Application Level)
Task:
Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:
E → E + E
E → E * E
E → id
Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:
Input:
id + id * id
Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

Q3. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers

2. **Compare** PEG with LL and LR parsing in terms of:
    - Grammar expressiveness
    - Ease of implementation
    - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

Q4. PEG vs CFG Parsing Strategy Evaluation

Task:
Design a PEG-based parser for the given expression grammar and critically evaluate its behavior compared to CFG-based parsing.

Grammar (CFG form):
E → E + T | T
T → T * F | F
F → id

Conditions:

- Convert CFG into PEG form
- Explain ordered choice behavior
- Demonstrate parsing steps
- Compare ambiguity handling with CFG
- Evaluate performance and backtracking
- Justify advantages and limitations

Test Case:
Input: id + id * id

Q5. Design a Hybrid Parsing Strategy

Task:
Design a hybrid parsing system that combines LL(1) and LR parsing techniques to efficiently parse a programming language with both simple and complex constructs.

Conditions:

- Identify which parts use LL(1) vs LR parsing
- Justify the hybrid design choice
- Define grammar transformations required
- Illustrate parsing workflow with diagram
- Demonstrate parsing of a sample input
- Evaluate advantages over standalone methods

Test Case:
Input: id + id * id

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

Name : [illegible]

Reg No : [illegible]

102 - CA1

Design Task

CSA 1440 — Compiler Design

1) Predictive Parser Design (LL(1))

Aim:

To construct an LL(1) predictive parsing table and parse the Input id+id*id

Grammar:

$E \rightarrow TE'$

$E' \rightarrow +TE' | \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \varepsilon$

$F \rightarrow (E) | id$

FIRST sets:

FIRST (E) = {(, id}

FIRST (E') = {+, ε}

FIRST (T) = {(, id}

FIRST (T') = {*, ε}

FIRST (F) = {(, id}

Follow sets:

FOLLOW (E) = {$,)}

FOLLOW (E') = {$,)}

FOLLOW (T) = {+, $,)}

FOLLOW (T') = {+, $,)}

FOLLOW (F) = {*, $,)}

Predictive parsing Table:

Non-terminal	id	+	*	(	)	$
E	TE'			TE'		
E'		+TE'			ε	ε
T	FT'			FT'		
T'		ε	*FT'		ε	ε
F	id			(E)		

parsing Trace:

I/P : id + id * id $

step	stack	Input	Action
1.	$E	id+id*id$	E->TE'
2.	$E'T	id+id*id$	T->FT'
3.	$E'T'F	id+id*id$	F->id
4.	$E'T'id	id+id*id$	match id
5.	$E'T'	+id*id$	T'->ε
6.	$E'	+id*id$	E'->+TE'
7.	$E'T+	+id*id$	match +
8.	$E'T	id*id$	T->FT'
9.	$E'T'F	id*id$	F->id
10.	$E'T'id	id*id$	match id

11	$E'T'	*id$	T'->*FT'
12	$E'T'F*	*id$	match*
13	$E'T'F	id$	F->id
14	$E'T'id	id$	match id
15	$E'T'	$	T'->ε
16	$E'	$	E'->ε
17	$	$	Accept

Result:

The input string id+id*id is accepted by the LL(1) predictive parser.

2) Shift Reducing parsing.

Aim:

To parse the input id+id*id using shift-reduce parsing and give multiplication higher precedence than addition.

Grammar:

E->E+E

E->E*E

E->id

Step	Stack	Input	Action
1	$	id+id*id$	shift id
2.	$id	+id*id$	Reduce E->id
3.	$E	+id*id$	shift +
4.	$E+	id*id$	shift id
5.	$E+id	*id$	Reduce E->id
6.	$E+E	*id$	shift *
7.	$E+E*	id$	shift id
8.	$E+E*id	$	Reduce E->id
9.	$E+E*E	$	Reduce E->E*E
10.	$E+E	$	Reduce E->E+E
11	$E	$	Accept

Expression Evaluation:

id + (id * id)

Result:

The input string is Accepted. The precedence rule ensures that multiplication is evaluated before addition.

Q3. Critically Evaluate the Effectiveness of Parsing Expression Grammars (PEG)

1. Introduction

Parsing Expression Grammar (PEG) is a modern grammar formalism used to describe the syntax of programming languages. It is an alternative to traditional Context-Free Grammar (CFG). PEG commonly uses recursive-descent parsing and introduces a feature called ordered choice.

Traditional LL parsers use top-down parsing, whereas LR parsers use bottom-up parsing. PEG provides a flexible way to define grammar rules and control how alternatives are selected.

2. Ambiguity Handling in PEG

A CFG may be ambiguous when an input string has more than one valid parse tree. For example, an arithmetic expression containing addition and multiplication may have different interpretations if operator precedence is not specified.

PEG handles alternatives through ordered choice. The parser tries the first alternative before considering the next one. Once an alternative succeeds, it does not automatically select another alternative simply because it matches more input.

This makes PEG's parsing behavior deterministic for a given grammar and input. However, grammar designers must arrange alternatives carefully to obtain the intended result.

3. Comparison of PEG, LL and LR Parsers

Feature	PEG	LL Parser	LR Parser
Parsing method	Commonly recursive descent	Top-down	Bottom-up
Ambiguity handling	Uses ordered choice	Requires a suitable LL grammar	Uses parsing tables and conflict-resolution rules
Grammar flexibility	Supports ordered alternatives and lookahead	Has restrictions based on lookahead	Supports many deterministic context-free grammars
Implementation	Often easy with PEG tools	Relatively simple	More complex to construct manually
Backtracking	May use backtracking	Standard LL(1) does not backtrack	Standard deterministic LR does not backtrack
Performance	Depends on grammar and implementation	Generally linear for LL(1)	Generally linear for deterministic LR
Memory	Packrat parsing may require more memory	Uses a stack and parsing table	Uses a stack and parsing tables

4. Real-World Application: Python

Python adopted a PEG-based parser in Python 3.9, replacing its previous LL(1)-based parser. The main reasons were:

- PEG provides greater flexibility for grammar design.
- It reduces restrictions associated with LL(1) parsing.
- It makes some language features easier to express.
- It simplifies the introduction of new syntax features.
- It supports ongoing development of the Python language.

Python's adoption demonstrates that PEG can be effective in real-world language development. However, it does not mean PEG is the best choice for every language.

5. Advantages of PEG

1. Ordered choice provides clear alternative-selection behavior.
2. Grammar rules can be readable and compact.
3. Lookahead helps distinguish similar syntax patterns.
4. It is useful for custom programming and domain-specific languages.
5. It supports flexible grammar design.

6. Limitations of PEG

1. Traditional PEG implementations may not support left recursion directly.
2. The order of alternatives can change parsing behavior.
3. Excessive backtracking can reduce performance.
4. Packrat parsing may consume significant memory.
5. Error reporting depends on the implementation.

7. Conclusion

PEG is an effective modern parsing technique for flexible and evolving languages. Its ordered-choice mechanism helps control ambiguity, and Python demonstrates its practical value. However, PEG is not suitable for every programming language. LL and LR parsers remain useful when their grammar requirements, performance characteristics, and existing tools better match the project.

Q4. PEG vs CFG Parsing Strategy Evaluation

1. Introduction

Parsing Expression Grammar (PEG) and Context-Free Grammar (CFG) are used to describe the syntax of programming languages. Both can represent arithmetic expressions, but they differ in how alternatives are interpreted.

CFG is widely used in traditional compiler design, while PEG uses ordered choice to determine how alternatives are tried.

2. Given Grammar

The CFG for arithmetic expressions is:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

Here, E represents an expression, T represents a term, F represents a factor, and id represents an identifier.
The grammar gives multiplication higher precedence than addition.

3. Conversion of CFG into PEG
The original grammar contains left recursion. This may cause problems for traditional recursive-descent PEG implementations. Therefore, the grammar can be rewritten as:
E <- T ("+" T)*
T <- F ("*" F)*
F <- "id"
In this grammar, * after a parenthesized expression means zero or more repetitions. The multiplication operator itself is represented by "*" inside the quoted string.
This form preserves the intended operator precedence: multiplication is handled at the term level, and addition is handled at the expression level.

4. Ordered Choice in PEG
PEG uses the / symbol for ordered choice.
For example:
A <- "a" / "ab"
The parser tries "a" first. If it succeeds, it does not automatically reconsider "ab" simply because that alternative matches more characters.
Therefore, alternative order is important. If the longer alternative must be preferred, "ab" should be placed before "a".
In the converted arithmetic grammar, precedence is controlled mainly by the structure of the rules rather than by ordered choice.

5. Parsing the Test Case
Input: id + id * id
The parser first recognizes the initial id as a factor and term. It then recognizes the addition operator, followed by another term. Within that term, it recognizes id * id as a multiplication expression.
The input is accepted and interpreted as:
\[id+(id*id) \]
Multiplication is grouped before addition because the PEG grammar places multiplication within the T rule.

6.PEG Compared with CFG-Based Parsing

Feature	**PEG**	**CFG with LL/LR**
Alternative selection	Ordered choice	Alternatives are unordered
Ambiguity	Parsing behavior is determined by choice order	A CFG may be ambiguous
Left recursion	Often needs rewriting or implementation support	Naturally supported by LR parsing
Parsing technique	Commonly recursive descent	LL uses top-down; LR uses bottom-up

Backtracking	May occur	Standard deterministic LL/LR parsing does not backtrack
Performance	Depends on grammar and implementation	Generally linear for fixed LL(1) and deterministic LR grammars
Implementation	Convenient with PEG libraries	LL is relatively simple; LR table construction can be complex

7. Advantages of PEG

1. Provides explicit alternative-selection behavior.
2. Supports readable and flexible grammar rules.
3. Uses repetition to represent repeated operators.
4. Supports lookahead for distinguishing syntax patterns.
5. Is useful for custom parsers and domain-specific languages.

8. Limitations of PEG

1. Alternative ordering may produce unexpected results.
2. Left recursion may need special handling.
3. Backtracking can affect performance.
4. Packrat parsing may require additional memory.
5. Error reporting depends on the parser implementation.

9. Conclusion

The CFG was converted into a PEG grammar that accepts id + id * id and interprets it as id + (id * id). PEG is useful for readable and flexible grammar design, while traditional CFG-based LL and LR parsers are useful for established compiler construction techniques. The choice depends on grammar complexity, performance, and implementation requirements.

Q5. Design a Hybrid Parsing Strategy Using LL(1) and LR Parsing

1. Introduction

A hybrid parsing strategy combines LL(1) and LR parsing techniques in one compiler. LL(1) is a top-down parsing method that is suitable for simple grammar structures. LR is a bottom-up parsing method that can handle many complex grammar structures, including left-recursive expression grammars.

Combining these methods can provide flexibility when different parts of a programming language have different parsing requirements.

2. Proposed Hybrid Strategy

The compiler divides parsing responsibilities between two components.

LL(1) parser:

- Handles suitable assignment statements and simple constructs.
- Uses FIRST and FOLLOW sets.
- Uses a predictive parsing table.
- Requires grammar rules that satisfy LL(1) conditions.

LR parser:

- Handles arithmetic expressions.
- Supports left-recursive grammar rules.
- Uses shift and reduce operations.
- Uses parsing tables to determine actions.
- Can enforce operator precedence and associativity.

3. Grammar and Transformations

Consider the grammar:

- S → id = E ;
- E → E + T | T
- T → T * F | F
- F → id | num

The expression grammar contains left recursion in E and T. This is acceptable for an LR parser but is not directly suitable for a conventional LL(1) parser.

If the expression grammar were assigned to LL(1), left recursion would need to be removed. For example:

- E → TE'
- E' → +TE' | ε
- T → FT'
- T' → *FT' | ε
- F → id | num

In the proposed hybrid design, the expression grammar remains left-recursive and is handled by LR. The LL(1) parser handles the statement structure.

4. Working Procedure

1. The lexical analyzer converts source code into tokens.
2. The parser dispatcher identifies the grammar section.
3. The LL(1) parser processes suitable simple statement structures.
4. When an arithmetic expression is encountered, the LR parser processes it.
5. The LR parser applies shift and reduce actions.
6. The parser combines the results into an Abstract Syntax Tree (AST).
7. Semantic analysis checks identifiers, declarations, and data types.
8. Syntax errors are reported with their locations.

5. Example

Consider the input expression:

id + id * id

The LR component parses the expression and groups multiplication before addition. The result is interpreted as:

\[id+(id*id) \]

In a complete compiler, the LL(1) component would handle the surrounding statement structure, while the LR component would handle the arithmetic expression.

6. Advantages of the Hybrid Strategy

1. Flexibility: Different grammar sections can use suitable parsing methods.
2. Efficiency: Each parser can be used where its characteristics are appropriate.
3. Modularity: The statement parser and expression parser can be developed separately.
4. Maintainability: Changes to one grammar section may be isolated from another.
5. Grammar support: LR can handle left-recursive expressions without requiring the same transformations as LL(1).
6. Reusability: Existing parser generators and libraries may be used for different components.

7. Limitations

1. Integrating two parsers increases implementation complexity.
2. The parsers must use consistent token definitions.
3. Grammar boundaries must be clearly defined.
4. Both components must produce compatible syntax-tree structures.
5. Error recovery across parser boundaries can be difficult.
6. A hybrid parser may be unnecessary for a small language that can be handled effectively by one parser.

8. Conclusion

A hybrid parsing strategy combines LL(1) parsing for suitable simple constructs with LR parsing for arithmetic expressions and other complex grammar structures. It offers flexibility and modularity, but it requires careful integration and additional development effort. It is most useful when different parts of a programming language have genuinely different parsing needs.